

O Acervo da Quarentena

Um catálogo do que sobrou de bonito no fim do mundo

NÍVEL

DIFÍCIL

acima do que você já praticou — é para esticar

STACK

Java puro (JDK 17+), sem bibliotecas externas, sem build tool

SOLUÇÃO EM

java/04-colecoes-streams/acervo-da-quarentena/src/

TEMPO ESTIMADO

3 a 6 horas, distribuídas — não tente numa sentada só

PRÉ-REQUISITO

o que você já fez: classes, encapsulamento, laços, `switch`

CONCEITOS NOVOS

interfaces e classes abstratas, polimorfismo, `enum`, coleções (`List`, `Map`, `Set`), `Comparator`, `equals / hashCode`, exceções customizadas

1. O contexto

Jackson, Wyoming. Vinte anos depois do surto. A cidade tem muros, tem gente, tem eletricidade algumas horas por dia — e tem uma sala nos fundos da escola onde se guarda tudo que as patrulhas resgatam das cidades mortas: partituras encharcadas, DVDs riscados, fitas de séries que ninguém mais vai gravar. Alguém precisa catalogar isso. Esse alguém é você.

"Você toca aquilo direito?" — perguntou a Maria, apontando pro violino no canto.

"Se eu tivesse a partitura inteira, tocava. Mas metade dos compassos virou mofo."

O acervo é heterogêneo por natureza: uma **partitura** não é um **filme**, que não é uma **série**. Mesmo assim, a comunidade quer perguntar a mesma coisa para todos eles: *quanto tempo isso ocupa?* Quando as luzes acendem por duas horas à noite, o que cabe nesse tempo — dois episódios, um filme, ou meia hora estudando escalas no violino?

É aí que mora o desafio: você vai construir um catálogo que trata coisas **diferentes** de forma **uniforme**. Esse é, literalmente, o problema que o polimorfismo existe para resolver. Até hoje você escreveu classes que vivem sozinhas (`Pessoa`, `Calculadora`). Agora você vai escrever uma *família* de classes.

2. O que você vai construir

Um programa de console, em Java puro, que modela o acervo e responde a quatro perguntas da comunidade. Nada de menu interativo obrigatório: a `Main` monta um acervo de exemplo, executa as operações e imprime os relatórios. (Menu com `Scanner` é bônus, no final.)

2.1 Os itens do acervo

Todo item catalogado, seja ele qual for, tem:

ATRIBUTO	TIPO	REGRA
codigo	texto	identificador único no acervo (ex.: PT-001 , FL-001 , SR-001). Nunca nulo, nunca vazio.
titulo	texto	nunca nulo, nunca vazio (nem só espaços).
ano	inteiro	entre 1500 e 2013 — depois disso ninguém mais publicou nada.
conservacao	enum	um entre: INTACTO , DESGASTADO , DANIFICADO , ILEGIVEL .

E cada tipo acrescenta o seu:

TIPO	ATRIBUTOS PRÓPRIOS	COMO SE CALCULA A DURAÇÃO, EM MINUTOS
Partitura	compositor, dificuldade (1 a 5), número de compassos, andamento em BPM	assuma 4 tempos por compasso: $(\text{compassos} \times 4) \div \text{bpm}$
Filme	diretor, duração em minutos	a própria duração
Série	número de episódios por temporada, duração média do episódio	$(\text{soma dos episódios de todas as temporadas}) \times \text{duração média}$

Repare no ponto crítico: **o acervo não pode ter um `if` perguntando "que tipo de item é este?"** para calcular a duração. Cada item sabe calcular a si próprio; o acervo apenas pergunta. Se você escrever `if (item instanceof Filme)` na classe do acervo, o desafio falhou no seu propósito — mesmo que o programa imprima os números certos.

2.2 O acervo

Uma classe que guarda os itens e oferece estas operações. As assinaturas exatas (nomes, tipos de retorno, checked vs. unchecked) são **decisão sua** — e você vai precisar defendê-las no code review.

OPERAÇÃO	RECEBE	DEVOLVE	FALHA QUANDO
catalogar	um item	—	já existe item com o mesmo <code>codigo</code> → erro de <i>item duplicado</i>
buscar por código	código	o item	não existe → sua escolha: exceção ou <code>Optional</code> . Justifique.
listar tudo	—	coleção de itens	nunca — mas quem recebe a lista não pode alterar o acervo por ela
agrupar por tipo	—	mapa: tipo → itens daquele tipo	nunca
duração total	—	minutos somados de todo o acervo	nunca
sessão da noite	minutos disponíveis	itens escolhidos, na ordem escolhida	minutos negativos → argumento inválido

2.3 A regra da sessão da noite

Duas horas de luz. O que cabe? A regra é **gulosa e determinística** — implemente exatamente assim, para que sua saída bata com a esperada:

1. Descarte os itens em estado **ILEGIVEL** — não dá para ler o que virou mofo.
2. Ordene os restantes por duração **decrescente**. Empate? Desempate por título em ordem alfabética.
3. Percorra a lista ordenada: inclua o item se ele couber no tempo *restante*; se não couber, pule e continue para o próximo.
4. Devolva os itens incluídos, na ordem em que foram incluídos.

Sim, guloso não dá a melhor combinação possível. Guarde essa incomodação: ela tem nome, e a gente conversa sobre isso no review.

3. Regras que separam o júnior do sênior

Estas não são detalhes. São o coração do desafio.

R1 — Um item inválido não deve conseguir nascer

Título vazio, ano em 2049, BPM zero ou negativo, dificuldade 9, duração negativa: o objeto **não pode ser construído**. Nada de criar e depois torcer para alguém validar lá na frente. Onde vive essa validação, e que tipo de erro ela sinaliza?

R2 — Dois itens são o mesmo item quando têm o mesmo código

Um `Filme` e uma `Partitura` com código `PT-001` são, para o acervo, o mesmo registro. Faça o acervo enxergar isso *sem* varrer a lista item a item comparando strings. Existe estrutura de dados para isso; existe também um par de métodos que você precisa sobrescrever para que ela funcione. Descubra quais — e o que acontece se você sobrescrever só um dos dois.

R3 — Quem recebe a lista não manda no acervo

Se alguém fizer `acervo.listarTudo().clear()`, o acervo **não** pode ficar vazio. Pesquise por "cópia defensiva" e "coleção imutável". Escolha um caminho e saiba dizer o que ele custa.

R4 — Erros do acervo são do acervo

Catalogar duplicado e buscar inexistente merecem exceções *suas*, com nome que conte a história, e não um `System.out.println("erro")` escondido dentro do método. Decida entre *checked* e *unchecked* — e tenha um motivo. "Foi o que apareceu no tutorial" não é motivo.

R5 — Uma ficha por item, escrita por ele mesmo

Cada item sabe se descrever em uma linha (veja o formato na seção 4). O acervo imprime; ele não monta o texto de cada tipo.

4. Exemplo — entrada e saída esperadas

4.1 Acervo de exemplo (monte-o na Main)

CÓDIGO	TIPO	TÍTULO	ANO	CONSERVAÇÃO	DADOS PRÓPRIOS
PT-001	Partitura	Prelúdio em Sol	1717	INTACTO	J. S. Bach · dif. 4 · 42 compassos · 60 bpm
PT-002	Partitura	Take On Me (arranjo p/ violino)	1985	DESGASTADO	a-ha · dif. 2 · 96 compassos · 84 bpm
PT-003	Partitura	Future Days	2013	ILEGIVEL	P. Yorn · dif. 3 · 64 compassos · 76 bpm
FL-001	Filme	Curtis e Viper 2	2013	INTACTO	dir. desconhecido · 118 min
SR-001	Série	Além da Imaginação	1959	DANIFICADO	temporadas [36, 29] episódios · 25 min/ep

4.2 Saída esperada

```
=== ACERVO DA QUARENTENA – JACKSON, WY ===
Itens catalogados: 5
Duração total: 1753,7 min (29h13)

--- POR TIPO ---
Partitura (3)
  [PT-001] Prelúdio em Sol (1717) · J. S. Bach · dif. 4/5 · 2,8 min · INTACTO
  [PT-002] Take On Me (arranjo p/ violino) (1985) · a-ha · dif. 2/5 · 4,6 min · DESGASTADO
  [PT-003] Future Days (2013) · P. Yorn · dif. 3/5 · 3,4 min · ILEGIVEL
Filme (1)
  [FL-001] Curtis e Viper 2 (2013) · dir. desconhecido · 118,0 min · INTACTO
Serie (1)
  [SR-001] Além da Imaginação (1959) · 2 temporadas · 65 episódios · 1625,0 min · DANIFICADO

--- SESSÃO DA NOITE (120 min disponíveis) ---
  [FL-001] Curtis e Viper 2 – 118,0 min
  Restam 2,0 min – nem o Bach cabe.
Total da sessão: 118,0 min de 120,0 min

--- SESSÃO DA NOITE (130 min disponíveis) ---
  [FL-001] Curtis e Viper 2 – 118,0 min
  [PT-002] Take On Me (arranjo p/ violino) – 4,6 min
  [PT-001] Prelúdio em Sol – 2,8 min
Total da sessão: 125,4 min de 130,0 min

--- TENTATIVAS INVÁLIDAS ---
Catalogar PT-001 de novo -> ItemDuplicadoException: código PT-001 já está no acervo
Buscar XX-999           -> ItemNaoEncontradoException: código XX-999 não existe
Partitura com 0 bpm     -> IllegalArgumentException: bpm deve ser maior que zero
```

Sobre os números — confira sua matemática

- PT-001: $(42 \times 4) \div 60 = \mathbf{2,8}$ min
- PT-002: $(96 \times 4) \div 84 = \mathbf{4,571...}$ → exibe **4,6**
- SR-001: $(36 + 29) \times 25 = \mathbf{1625,0}$ min
- PT-003: $(64 \times 4) \div 76 = \mathbf{3,368...}$ → exibe **3,4** (entra no total, mas nunca na sessão da noite)
- Total: $2,8 + 4,571... + 3,368... + 118 + 1625 = \mathbf{1753,7}$ min

Exiba sempre com **uma casa decimal e vírgula** (padrão pt-BR). Mas **não arredonde antes de somar** — some os valores cheios e arredonde só na hora de imprimir. Se você arredondar cedo, o total vai fechar diferente. Isso não é frescura de professor: é a categoria de bug que derruba fechamento contábil em produção.

5. Antes de escrever a primeira linha de código

Não abra o editor ainda. Corrigir um plano custa cinco minutos; corrigir código já escrito custa uma tarde. Me responda por escrito — pode ser num `PLANO.md` dentro da pasta do desafio:

1. **O problema, com suas palavras.** Sem repetir o enunciado. O que este programa precisa fazer, em três frases suas?
2. **Seu desenho.** Quais classes vão existir? Quem herda de quem, ou quem implementa o quê? Por que você escolheu classe abstrata *ou* interface — e o que a outra opção te daria de diferente?
3. **Suas estruturas de dados.** O que guarda os itens dentro do acervo: `ArrayList` ? `HashMap` ? `HashSet` ? Escolha uma e diga o que ela te dá de graça e o que ela te cobra. Dica de rumo: pense em qual operação do acervo é a mais frequente.

Me traga esse plano *antes* de codar. Se tiver furo grave, eu pergunto — não conserto.

6. Critérios de aceite

Está pronto quando **todos** estes forem verdade:

- ☐ Compila e roda com `javac *.java && java Main`, sem dependência externa.
- ☐ Existe um tipo comum (interface ou classe abstrata) que os três tipos de item cumprem.
- ☐ O cálculo de duração é polimórfico: `zero instanceof` e `zero switch` por tipo dentro da classe do acervo.
- ☐ O estado de conservação é um `enum`, não uma `String` solta.
- ☐ Construtores rejeitam dados inválidos (R1), com mensagem que diz o que está errado.
- ☐ Catalogar código duplicado lança uma exceção sua, com mensagem útil.
- ☐ Buscar código inexistente tem comportamento definido e documentado por você.
- ☐ Alterar a coleção devolvida pelo acervo **não** altera o acervo (R3).
- ☐ O agrupamento por tipo devolve um `Map`.
- ☐ A sessão da noite segue a regra gulosa da seção 2.3, à risca, e ignora `ILEGIVEL`.
- ☐ A saída reproduz os números do exemplo 4.2 (formato pode variar; os números, não).
- ☐ A `Main` demonstra também os três casos de erro da seção 4.2 — com o programa continuando vivo depois de cada um.
- ☐ Nomes em português ou inglês, mas **consistentes** no projeto inteiro. Escolha um idioma e não misture (olho no seu `classificacaoDeImc()` convivendo com `getName()`).

Bônus — só depois que o principal estiver de pé

- Refaça *agrupar por tipo* e *duração total* com Stream API (`Collectors.groupingBy`, `mapToDouble`). Compare com sua versão de laço: qual ficou mais legível? Guarde as duas para o review.
- Faça os itens implementarem `Comparable` e discuta: ordem natural ou `Comparator` externo?
- Menu interativo com `Scanner`, tratando entrada não numérica sem quebrar.

- Um método `autoTeste()` na `Main` com `assert` ou `if` + mensagem, conferindo os números da seção 4.2 sozinho.

O que não vale

- Pedir a solução pronta ao Claude (o repositório está em Modo Yoda — ele vai te devolver perguntas). Dúvida conceitual, erro de compilação, "o que é uma interface": *pode* perguntar, e deve.
- Colar código de fórum que você não sabe explicar linha a linha. No review, eu pergunto.
- Bibliotecas externas. JDK puro.
- Um arquivo gigante com tudo dentro. Uma classe por arquivo.

7. Por que este desafio existe

Você já sabe fazer um programa funcionar. Este desafio é sobre a próxima camada: fazer um programa **aceitar mudança**. Daqui a duas semanas, a comunidade resgata uma caixa de discos de vinil e quer catalogar áudio também. Se o seu desenho estiver certo, você acrescenta uma classe e não toca em *nenhuma* linha do acervo. Se estiver errado, você vai abrir `if` s em quatro lugares diferentes e esquecer o quinto.

Isso tem nome no mercado — Princípio Aberto/Fechado, o "O" de SOLID — e é exatamente o que uma banca sênior procura quando propõe um take-home aparentemente bobo de "catálogo de produtos". O júnior entrega funcionando. O sênior entrega funcionando e, quando perguntam "e se aparecer um tipo novo?", responde sorrindo. É também o mesmo padrão por trás de `List`, `Map` e do `Comparator` que você vai usar aqui: contrato no meio, implementações trocáveis nas pontas. Quando você chegar no Spring, vai reencontrar essa mesma ideia em cada `@Repository` que você injeta por interface.

Vai errar. É pra errar. O erro é o professor mais honesto que você vai ter — ele nunca mente para agradar. Quando travar, me traga o que você tentou, não o que você quer que eu escreva.